```
import pandas as pd
import re
import nltk
from collections import defaultdict, Counter

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
df = pd.read_csv("Twitter_Data.csv")
print(df.head())
```

```
                                          clean_text  category
0  when modi promised “minimum government maximum...      -1.0
1  talk all the nonsense and continue all the dra...       0.0
2  what did just say vote for modi  welcome bjp t...       1.0
3  asking his supporters prefix chowkidar their n...       1.0
4  answer who among these the most powerful world...       1.0
```

```
def clean_tweet(text):
    # Ensure the input is treated as a string, handling non-string types gracefully
    text = str(text) if not isinstance(text, str) else text
    text = re.sub(r"http\S+", "", text)   # remove URLs
    text = re.sub(r"@\w+", "", text)      # remove mentions
    text = re.sub(r"#\w+", "", text)      # remove hashtags
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    return text.lower()

df["clean_text"] = df["clean_text"].apply(clean_tweet)
```

```
import nltk

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('tagsets')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package tagsets to /root/nltk_data...
[nltk_data]   Package tagsets is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
True
```

```
tagged_sentences = []

for tweet in df["clean_text"].head(500):
    tokens = nltk.word_tokenize(tweet)
    tags = nltk.pos_tag(tokens)
    if tags:
        tagged_sentences.append(tags)

print(tagged_sentences[0])
```

```
[('when', 'WRB'), ('modi', 'NN'), ('promised', 'VBD'), ('minimum', 'JJ'), ('government', 'NN'), ('maximum', 'JJ'), ('governanc
```

```
nltk.data.path.append('/usr/local/share/nltk_data')
```

```
transition = defaultdict(Counter)

for sent in tagged_sentences:
    for i in range(len(sent)-1):
        t1 = sent[i][1]
        t2 = sent[i+1][1]
```

```
        transition[t1][t2] += 1
```

```
transition_prob = {}

for tag in transition:
    total = sum(transition[tag].values())
    transition_prob[tag] = {t: c/total for t,c in transition[tag].items()}

print(list(transition_prob.items())[:3])
```

```
[('WRB', {'NN': 0.23958333333333334, 'VBZ': 0.010416666666666666, 'JJS': 0.010416666666666666, 'JJ': 0.20833333333333334, 'VBN
```

```
emission = defaultdict(Counter)

for sent in tagged_sentences:
    for word, tag in sent:
        emission[tag][word] += 1

emission_prob = {}
for tag in emission:
    total = sum(emission[tag].values())
    emission_prob[tag] = {w: c/total for w,c in emission[tag].items()}
```

```
word_freq = Counter()

for sent in tagged_sentences:
    for w,t in sent:
        word_freq[w] += 1

rare = [w for w in word_freq if word_freq[w] == 1]
print("Rare words:", rare[:10])
```

```
Rare words: ['maximum', 'difficult', 'reforming', 'temples', 'drama', 'main', 'relax', 'prefix', 'names', 'service']
```

```
for tag in list(transition_prob.keys())[:3]:
    print(tag, transition_prob[tag])
```

```
WRB {'NN': 0.23958333333333334, 'VBZ': 0.010416666666666666, 'JJS': 0.010416666666666666, 'JJ': 0.20833333333333334, 'VBN': 0.
NN {'VBD': 0.061129090255303845, 'JJ': 0.05537576411362819, 'VBG': 0.02481121898597627, 'WRB': 0.011506652283351312, 'NN': 0.3
VBD {'JJ': 0.21070234113712374, 'PRP': 0.026755852842809364, 'RB': 0.09364548494983277, 'WP': 0.010033444816053512, 'NN': 0.17
```

```
test = "love this movie"
tokens = nltk.word_tokenize(test)
print(tokens)
```

```
['love', 'this', 'movie']
```

```
# V1(tag) = P(tag) * P(word | tag)
```

```
# V2(tag2) = max [ V1(tag1) * P(tag2|tag1) * P(word2|tag2) ]
```

```
emission_prob
transition_prob
```

```
 'RB': 0.038461538461538464
```

```
  'RB': 0.038461538461538464,
  'JJ': 0.4230769230769231,
  'WDT': 0.038461538461538464,
  'VB': 0.038461538461538464,
  'PRP': 0.038461538461538464,
  'PRP$': 0.038461538461538464,
  'VBD': 0.038461538461538464},
 'CD': {'NN': 0.2857142857142857,
  'CC': 0.02040816326530612,
  'NNS': 0.3673469387755102,
  'RB': 0.061224489795918366,
  'DT': 0.061224489795918366,
  'JJ': 0.10204081632653061,
  'VBZ': 0.04081632653061224,
  'MD': 0.02040816326530612,
  'JJR': 0.02040816326530612,
  'PRP': 0.02040816326530612},
 'FW': {'VBZ': 0.16666666666666666,
  'NN': 0.2777777777777778,
  'RB': 0.1111111111111111,
  'PRP$': 0.16666666666666666,
  'IN': 0.05555555555555555,
  'VBD': 0.16666666666666666,
  'FW': 0.05555555555555555},
 'RP': {'JJS': 0.04,
  'JJ': 0.32,
  'RB': 0.08,
  'PRP$': 0.08,
  'NN': 0.24,
  'VB': 0.04,
  'NNS': 0.08,
  'CC': 0.04,
  'IN': 0.04,
  'DT': 0.04},
 'WP$': {'NN': 1.0},
 'NNP': {'NN': 0.6666666666666666, 'JJ': 0.3333333333333333},
 'TO': {'VB': 1.0}}
```